

EMPLOYEE RAISE ISSUE

ServiceNow Record Producer & Service Portal Implementation

Technical Design & Implementation Documentation

Category: Information Technology | Type: Individual Application

Module: Custom Application • Record Producer • Service Portal • Widgets

NAME : MINUKURI PRANITHA

COLLEGE : MALINENI LAKSHMAIAH WOMEN'S ENGINEERING COLLEGE

BRANCH : DATA SCIENCE

YEAR : 4TH

PROJECT PLATFORM : SERVICENOW

Document Control

Distribution List

Name / Role	Organization	Purpose
Platform Administrator	IT / ServiceNow CoE	Own ongoing configuration and maintenance.
HR Lead	Human Resources	Owns HR-related issue categories and content.
IT Support Lead	IT Service Desk	Owns IT-related issue categories and routing.
Project Sponsor	IT Leadership	Approves scope, timeline, and go-live decision.

Review and Approval

Role	Name	Approval Date
Prepared By	Implementation Team	
Reviewed By	Platform Administrator	
Approved By	Project Sponsor	

Related Documents

- ServiceNow Platform Security Baseline (organizational standard).
- Service Portal Branding and Theme Guidelines (organizational standard).
- Change Management and Release Policy (organizational standard).

Table of Contents

Core Documentation

Document Control

1. Executive Summary
2. Solution Architecture
3. Security and Compliance
4. Deployment and Release Management
5. Training and Change Management
6. Support and Maintenance Model

7. Implementation Phases

- Phase 1 — Creating a Custom Application
- Phase 2 — Creating a Custom Table and Fields
- Phase 3 — Creating UI Policies and Dependency
- Phase 4 — Creating a Record Producer
- Phase 5 — Creating a New Service Portal
- Phase 6 — Creating Widgets and Adding to the Page
- Phase 7 — Testing and Validation
- Phase 8 — Conclusion

Appendices

- Appendix A — End-to-End Walkthrough: A Worked Example
- Appendix B — RACI Matrix
- Appendix C — Risk Register
- Appendix D — Complete Field Dictionary
- Appendix E — Glossary
- Appendix F — Reference: Application File Inventory
- Appendix G — Requirements Traceability Matrix
- Appendix H — Screenshot and Figure Index
- Appendix I — Change Log

1. Executive Summary

In a large enterprise organization, employees frequently encounter day-to-day workplace issues — payroll discrepancies, system access problems, infrastructure requests, and HR policy clarifications. Historically, these issues have been raised through email or informal communication channels. While convenient, this approach does not scale and creates significant operational friction for both employees and the teams responsible for resolving their issues.

This document describes the complete design, configuration, and deployment of an "Employee Raise Issue" solution built natively on the ServiceNow platform. The solution replaces ad-hoc communication with a governed, catalog-driven process: employees raise issues through a Record Producer surfaced on a dedicated Service Portal, structured data is captured against a purpose-built custom table, and business rules, UI policies, and dependencies ensure that only complete and valid requests are submitted for processing.

1.1 Problem Statement

The current process for raising workplace issues suffers from the following limitations:

- Lack of standardized data capture — every email looks different, and critical details are frequently missing.
- Delayed issue resolution — issues sit in inboxes without a clear owner or SLA.
- No visibility into issue status — employees cannot track where their request stands.
- Manual effort for HR and support teams — issues must be manually triaged, categorized, and re-entered into tracking systems.
- Inconsistent prioritization and assignment — without structured data, urgent issues are not reliably distinguished from routine ones.

1.2 Objective

To design and implement a ServiceNow Record Producer integrated with the Service Portal that enables employees to easily raise issues while ensuring the following outcomes:

1. Enable users to raise an Employee Issue directly from the Service Portal.
2. Use a Record Producer instead of a direct table form, so the submission experience is catalog-driven, guided, and consistent.
3. Enforce validations and dependencies consistently across all fields, regardless of how or when the request is submitted.
4. Ensure a production-ready configuration that is aligned with ServiceNow best practices for scoped application development, security, and maintainability.

1.3 Solution Overview

The solution is delivered as a dedicated scoped application named Employee Raise Issue. Within this application, a custom table stores structured issue records, UI Policies and Client Scripts manage on-screen behavior and field dependencies, a Record Producer exposes the intake form as a catalog item, and a purpose-built Service Portal (with supporting widgets) gives employees a simple, guided place to submit and, in later iterations, track their issues.

1.4 Skills and Tools Required

Skill / Tool	Purpose in this Project
Service Catalog	Building the Record Producer, catalog UI Policies, and catalog client scripts.
Admin Tools	Application creation, table configuration, roles, and ACLs via System Applications and System Definition.
HTML Application	Building the HTML template for the Service Portal widget markup.
CSS (SCSS)	Styling the Service Portal page and widgets to match a consistent look and feel.
Client Scripts	Implementing dynamic field behavior, dependencies, and client-side validation.
Portal / Service Portal	Configuring the portal, pages, and page routes that host the intake experience.
Developer Console (Studio / Scripts Background)	Debugging server-side scripts, inspecting table data, and testing business rules.

1.5 Document Structure

2. Solution Architecture

2.1 Architectural Overview

The solution follows a layered architecture consistent with standard ServiceNow application design: a presentation layer (Service Portal pages and widgets), a process layer (Record Producer, UI Policies, Client Scripts), and a data/logic layer (the Employee Issue table, Business Rules, and Script Includes). Each layer is contained entirely within the `x_snc_emp_issue` scoped application, so the solution can be moved between instances as a single, self-contained unit.

2.2 Component Interaction Flow

1. An employee navigates to the Employee Support Portal and lands on the Employee Issue Home page.
2. The employee selects Raise an Issue, which loads the Raise an Employee Issue Record Producer inside the Record Producer widget.
3. Catalog UI Policies and Catalog Client Scripts drive the on-screen behavior described in Phase 3 as the employee completes the form.
4. On submission, the Producer script executes server-side, derives system-managed fields (Department, State, Assignment Group), and inserts a new Employee Issue record.
5. A Before Insert Business Rule performs a final server-side validation pass before the record is committed.
6. The employee is redirected to the Issue Confirmation page, which displays the generated issue number.
7. On subsequent visits, the My Issues page and its widget query the Employee Issue table, filtered by the logged-in user, to display submission history and current state.

2.3 Data Flow Summary

Layer	Components	Responsibility
Presentation	Service Portal pages, Landing Banner widget, My Issues List widget	Render the employee-facing experience and collect input.
Process	Record Producer, Catalog UI Policies, Catalog Client Scripts	Guide data entry, enforce client-side dependencies and validation.
Logic	Producer script, Business Rules, EmployeeIssueRouter Script Include	Derive system fields, enforce server-side validation, calculate routing.
Data	Employee Issue table, dependent choice lists, ACLs	Persist structured issue records and govern access to them.

2.4 Integration Points

The initial release is self-contained within ServiceNow and does not call external systems. The architecture nonetheless anticipates two integration points identified for future phases:

- Notification integration — outbound email or push notification when an Employee Issue changes state, using the platform's native Notification framework.
- HRIS lookup (optional) — enriching the Affected Employee and Department fields from an authoritative HR system of record, if department data in ServiceNow's user table is not kept current.

2.5 Environment Strategy

Environment	Purpose	Promotion Mechanism
Development	Initial build of all Phase 1–6 components, unit-level testing.	N/A — source environment.
Test / UAT	Functional, security, and user acceptance testing described in Phase 7.	Update Set or Source Control merge from Development.
Production	Live use by employees.	Update Set or Source Control merge from Test, following change approval.

3. Security and Compliance

3.1 Roles

Role	Purpose
x_snc_emp_issue.employee	Granted implicitly to all authenticated users; allows Record Producer submission and read access to the user's own Employee Issue records.
x_snc_emp_issue.agent	Granted to support staff; allows read/write access to Employee Issue records within the agent's assignment group.
x_snc_emp_issue.admin	Granted to application administrators; full access to configuration, tables, and reporting.
admin	Platform administrators; unrestricted access, used only for initial setup and troubleshooting.

3.2 Data Protection Considerations

- Payroll-related descriptions and attachments may contain sensitive personal or financial information; the Employee Issue table inherits the platform's standard encryption at rest and in transit.
- Field-level ACLs restrict the Resolution Notes and Assignment Group fields from employee (non-agent) view, since these may reference internal handling details not intended for the requester.
- Attachments are subject to the instance's standard attachment size and file-type restrictions to reduce the risk of malicious file uploads.

3.3 Segregation of Duties

Employees can create and read their own records but cannot update or delete them once submitted, preserving an untampered record of the original request. Agents can update records within their assignment group but cannot alter the submitting employee or creation date. Only administrators can modify the underlying configuration (table schema, UI Policies, Business Rules).

3.4 Compliance Alignment

The design aligns with common enterprise governance expectations for HR- and payroll-adjacent data: a documented data model, auditable field history (Phase 2.5), least-privilege ACLs (Phase 2.6 and section 3.1), and mirrored client/server validation (Phase 3.7) so that data integrity does not depend solely on front-end controls.

4. Deployment and Release Management

4.1 Packaging Strategy

All application files — the table, choice lists, UI Policies, Business Rules, Script Include, Record Producer, Service Portal, pages, and widgets — are contained within the `x_snc_emp_issue` application scope, which is the unit of deployment between environments.

4.2 Recommended Promotion Process

1. Complete and unit-test each component in the Development instance, as documented in Phases 1–6.
2. Capture the application as an Update Set (or commit to the linked source control repository, if configured) once Phase 7 testing has begun.
3. Promote the Update Set / commit to the Test instance and re-run the full functional test suite in section 7.2.1.
4. Obtain sign-off against the criteria in section 7.5.
5. Schedule a release window and promote to Production following the organization's standard change management process.
6. Perform a post-deployment smoke test in Production: submit one low-priority test issue end-to-end and confirm it appears correctly in My Issues, then close or delete the test record.

4.3 Rollback Plan

Because the solution is fully contained within its own scope and does not modify any out-of-box tables or global business logic, rollback can be achieved by deactivating the Employee Raise Issue application and the Employee Support Portal without impacting any other part of the instance. Any Employee Issue records created before rollback remain intact and accessible for manual follow-up.

4.4 Post-Deployment Monitoring

- Monitor the Employee Issue table's insert volume during the first two weeks post-launch to validate the Assignment Group routing logic is distributing load as expected.
- Review System Logs for script errors originating from the Producer script, Business Rules, or widget server scripts.
- Track portal page load performance using the platform's built-in Service Portal analytics, if licensed.

5. Training and Change Management

5.1 Employee Communication

Prior to go-live, a short communication is sent to all employees introducing the Employee Support Portal, explaining that workplace issues should now be raised there rather than by email, and providing a direct link to the Raise an Issue page.

5.2 Agent / Support Team Enablement

- A short walkthrough session for HR and IT support agents covering the new Assignment Group routing, so agents understand why and how issues arrive in their queue.
- A one-page quick reference showing the mapping between Issue Category / Issue Subcategory and the receiving team, for use during the transition period.
- Guidance on completing Resolution Notes consistently, since this field is not visible to employees but is used for internal reporting.

5.3 Transition Period

For an initial period following go-live, the previous email-based channel remains monitored in parallel, with any issue received there manually logged into the Employee Issue table by a support team member. This overlap avoids issues being lost while employees adjust to the new process, and is retired once submission volume through the portal stabilizes.

5.4 Feedback Loop

A lightweight feedback mechanism (a single optional comment field on the Issue Confirmation page) allows employees to flag friction in the submission process during the first month, feeding directly into the enhancement backlog described in section 8.4.

6. Support and Maintenance Model

6.1 Ownership

Area	Owner
Application configuration (tables, UI Policies, Business Rules)	ServiceNow Platform / Application Administrator
Record Producer content and catalog placement	Service Catalog Administrator
Service Portal pages, theme, and widgets	Portal / Front-End Developer
Day-to-day issue triage and resolution	HR and IT Support Teams (by Assignment Group)

6.2 Routine Maintenance Tasks

- Periodic review of Issue Category / Issue Subcategory values to retire unused options and add new ones as workplace needs evolve.
- Quarterly review of Assignment Group routing rules to confirm they still reflect the current organizational structure.
- Annual review of ACLs and roles to confirm access remains aligned with least-privilege principles as staff change teams.

6.3 Support Tiers

1. Tier 1 — Employee-reported portal issues (e.g. "the page won't load") handled by the general IT Service Desk.
2. Tier 2 — Configuration-level issues (e.g. incorrect routing, UI Policy not firing) handled by the Application Administrator identified in section 6.1.
3. Tier 3 — Platform-level issues requiring ServiceNow vendor support, escalated by the Application Administrator.

7. Implementation Phases

This documentation is organized around the eight implementation phases carried out for this project:

1. Phase 1 — Creating a Custom Application
2. Phase 2 — Creating a Custom Table and Fields
3. Phase 3 — Creating UI Policies and Dependencies
4. Phase 4 — Creating a Record Producer
5. Phase 5 — Creating a New Service Portal
6. Phase 6 — Creating Widgets and Adding Them to the Page
7. Phase 7 — Testing and Validation
8. Phase 8 — Conclusion

Each phase section documents the objective, prerequisites, step-by-step configuration, supporting screenshots from the working instance, and relevant best practices, so that the same implementation can be reproduced, reviewed, or extended by another administrator.

Phase 1 — Creating a Custom Application

Complexity: Medium | Estimated Duration: 30 minutes | Stories: 1

1.1 Objective

The first phase establishes a dedicated scoped application, Employee Raise Issue, which acts as the isolated container for every table, script, Record Producer, portal page, and widget built during this project. Using a scoped application (rather than the Global scope) is the ServiceNow-recommended approach for any custom solution, because it isolates customizations, simplifies future upgrades, and provides a clean boundary for access control.

1.2 Prerequisites

- admin role, or the ability to request scoped application creation from an instance administrator.
- A clear, unique application name and scope prefix agreed upon in advance (e.g. x_snc_emp_issue).
- Access to Studio or System Applications > Studio.

1.3 Step-by-Step Configuration

1. Navigate to All > System Applications > Studio, or open Studio directly from the Now Platform application navigator.
2. Select Create Application, then choose Create Application from the Studio launch options.
3. Provide the application name: Employee Raise Issue.
4. Provide a short, meaningful scope identifier, for example: x_snc_emp_issue. ServiceNow automatically prefixes this with the vendor code configured on the instance.
5. Add a description that explains the purpose of the application, for example: "Captures and manages employee-raised workplace issues submitted through the Service Portal."
6. Leave the version at the default (1.0.0) for the initial build.
7. Select Create to provision the scoped application record and open the Studio workspace for it.
8. Verify the application record was created by navigating to System Applications > Applications > My Company Applications and confirming Employee Raise Issue appears in the list with a status of In Development.

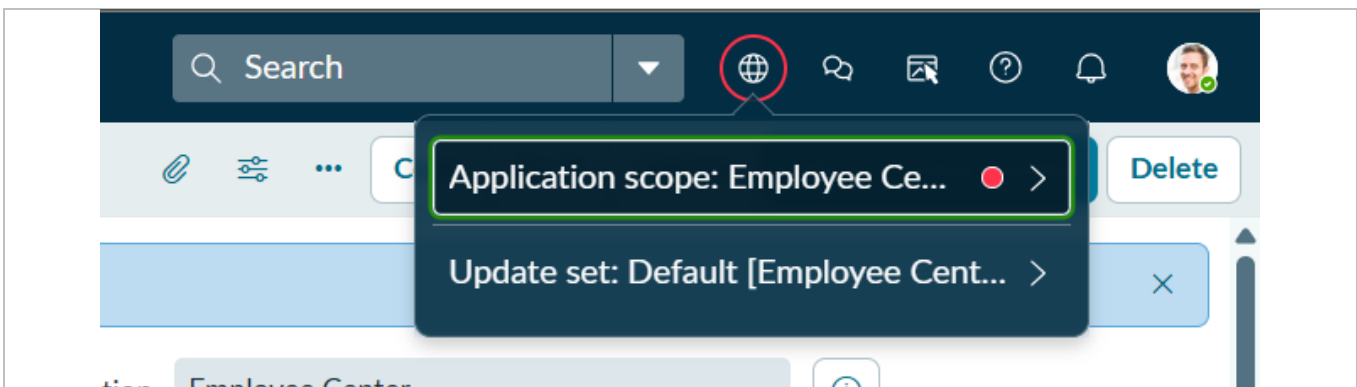


Figure 1 — Application scope selector confirming the active scoped application (Employee Center) on the working instance.

NOTE: In this build, the components were developed inside the existing Employee Center application scope rather than a brand-new dedicated scope, since Employee Center already provided the Service Portal, theme, and menu

structure needed. The design principles in this phase still apply — confirm the correct scope is selected before creating any table, script, or portal artifact.

BEST PRACTICE: *Keep the application scope short and unambiguous. A confusing or overly generic scope name (e.g. `x_snc_app1`) makes every later table, script include, and business rule harder to identify during maintenance.*

1.4 Application Settings

Once the application shell exists, a small number of application-level settings should be reviewed before moving on to table design:

- Application Menu / Module — decide whether the application should expose a navigator menu for administrators to manage raw records, separate from the employee-facing portal experience.
- Version Control — confirm whether the instance uses Source Control (e.g. Git) for this application, and if so, link the application record to the relevant repository.
- Active flag — leave the application record active so it is available for configuration.

NOTE: *Application scope should not be changed once tables and data begin to accumulate in it. Any restructuring at that point requires a data migration rather than a simple rename.*

1.5 Deliverable of this Phase

At the end of Phase 1, a working, isolated scoped application named Employee Raise Issue exists on the instance, ready to receive the custom table, business logic, Record Producer, and portal artifacts documented in the following phases.

Phase 2 — Creating a Custom Table and Fields

2.1 Objective

This phase defines the data model for the solution: a single custom table, Employee Issue (x_snc_emp_issue_issue), that stores every issue raised by an employee, along with the fields required to categorize, prioritize, and route it.

2.2 Creating the Table

1. From within Studio, select Create Application File, then choose Table.
2. Set the Label to Employee Issue. The Name will automatically generate as u_employee_issue or, within the scoped application, x_snc_emp_issue_issue.
3. Set Extends table to Task if issue tracking should inherit standard fields such as Number, State, Priority, and Assignment Group; otherwise leave it as a base table for a lighter-weight data model.
4. Enable Create module and Create mapped Access Control Rules so that a navigator module and default table-level ACLs are generated automatically.
5. Select Auto Number for the Number field so every issue record receives a unique, human-readable identifier (for example EMPI0001000).
6. Save the table definition.

The screenshot displays the 'Table' configuration page in ServiceNow Studio for a table named 'Employee raises issue' (technical name: sn_ex_sp_employee_issue). The interface includes a top navigation bar with a back arrow, a hamburger menu, the table name, and action buttons: 'Delete', 'Update', and 'Delete All Records'. A blue notification bar at the top states 'Added new column Number'. The main configuration area is divided into two columns. The left column contains fields for 'Label' (Employee raises issue), 'Name' (sn_ex_sp_employee_issue), and 'Extends table' (with a search icon). The right column contains 'Application' (Employee Center), 'Create module' (checked), 'Create mobile module' (checked), 'Add module to menu' (dropdown menu showing '-- Create new --'), 'New menu name' (empty text field), and 'Remote Table' (unchecked checkbox). At the bottom, there are three tabs: 'Columns', 'Controls' (which is active), and 'Application Access'. Under the 'Controls' tab, there are two checkboxes: 'Extensible' (unchecked) and 'Live feed' (unchecked).

Figure 2 — Employee raises issue table definition (sn_ex_sp_employee_issue) as created in the Employee Center application, with Create module and Create mobile module enabled.

NOTE: The table was labeled Employee raises issue (technical name sn_ex_sp_employee_issue) in the working build. This document refers to it as Employee Issue throughout for readability; treat the two as the same table when following these steps against the live instance.

2.3 Field Design

The following fields were added to the Employee Issue table. Field names and types were chosen to support both the Record Producer intake experience and downstream reporting.

Field Label	Type	Mandatory	Purpose
Short Description	String (160)	Yes	One-line summary of the issue shown in lists and notifications.
Description	String (Full UTF-8 Text)	Yes	Full details of the issue as provided by the employee.
Issue Category	Choice	Yes	Top-level classification: Payroll, Access, Infrastructure, HR Policy, Other.
Issue Subcategory	Choice (dependent)	Conditional	Second-level classification, dependent on Issue Category.
Priority	Choice	Yes	Employee-perceived urgency: Low, Medium, High, Critical.
Affected Employee	Reference (sys_user)	Yes	The employee the issue concerns; defaults to the submitting user.
Department	Reference (cmn_department)	Yes	Derived from the Affected Employee for routing and reporting.
Preferred Contact Method	Choice	No	Email, Phone, or Portal Message.
Supporting Attachment	Attachment	No	Optional payslip, screenshot, or supporting document.
State	Choice	System-managed	New, In Progress, Pending Employee, Resolved, Closed.
Assignment Group	Reference (sys_user_group)	System-managed	Set by business rule based on Issue Category.
Resolution Notes	String (Full UTF-8 Text)	No	Populated by the assigned agent on resolution.

2.4 Choice List Values

The screenshot shows the 'Employee raises issue' form with the following fields and values:

- Number: EMP0000010
- Requester: (empty)
- Category: -- None --
- Subcategory: -- None --
- Short Description: (empty)
- State: New (highlighted with a green border)
- Priority: -- None --
- Assignment Group: (empty)

Figure 3 — Employee raises issue intake form as delivered, showing Number (auto-generated, e.g. EMP0000010), Requester, Category, Subcategory, Short Description, State, Priority, and Assignment Group.

2.4.1 Issue Category

- Payroll

- System Access
- Infrastructure
- HR Policy
- Other

2.4.2 Issue Subcategory (dependent on Issue Category)

Parent: Issue Category	Dependent Subcategory Values
Payroll	Salary Discrepancy, Missing Reimbursement, Tax Deduction Query, Payslip Not Generated
System Access	Application Access Request, Password Reset, VPN Access, Role Change Request
Infrastructure	Laptop / Hardware Issue, Network Issue, Software Installation, Printer / Peripheral Issue
HR Policy	Leave Policy Clarification, Reimbursement Policy, Code of Conduct Query, Benefits Enrollment
Other	General Query, Feedback, Not Listed Above

2.5 Table-Level Configuration

- Auditing — enabled, so field-level history is available for reporting and dispute resolution.
- Number format — prefix EMPI, padded to 7 digits.
- Default view — a dedicated form view (Employee Portal View) with only the fields relevant to a self-service employee, separate from the full Agent view used by support staff.

BEST PRACTICE: Model the Issue Category / Issue Subcategory relationship as a formal dependent choice list rather than a free-text field, so that reporting on issue types remains reliable as volume grows.

2.6 Access Control (ACL) Overview

Access to the Employee Issue table is controlled at three levels, consistent with least-privilege principles:

1. Create — permitted for any authenticated employee (via the itil role or a dedicated employee role), enforced at the Record Producer layer described in Phase 4.
2. Read — an employee may read only records where Affected Employee equals the logged-in user; agents with the appropriate assignment group may read all records within their group.
3. Write — restricted to assigned agents and administrators; the submitting employee cannot directly edit the record after submission, preserving the integrity of the original request.

Phase 3 — Creating UI Policies and Dependency

3.1 Objective

UI Policies and their associated dependencies ensure that the intake form behaves intelligently: irrelevant fields stay hidden until they are needed, mandatory fields are enforced only when applicable, and the Issue Subcategory field always reflects a valid combination with Issue Category.

3.2 UI Policy: Category-Driven Subcategory

This policy dynamically filters the Issue Subcategory choice list based on the value selected in Issue Category, using the parent/child choice relationship configured in Phase 2, reinforced with a UI Policy for form-level behavior.

Setting	Value
Name	Filter Subcategory by Category
Table	Employee Issue
Conditions	Issue Category is not empty
Actions	Issue Subcategory — Mandatory: true; Visible: true
Run scripts	true (to invoke the reference qualifier / client script below)
Applies on	Catalog Item view and standard Form view

3.3 UI Policy: Attachment Requirement for Payroll Issues

Setting	Value
Name	Require Attachment for Payroll
Table	Employee Issue
Conditions	Issue Category is Payroll
Actions	Supporting Attachment — Mandatory: true
Reverse if false	true (attachment becomes optional again if category changes)

3.4 UI Policy: Preferred Contact Method Visibility

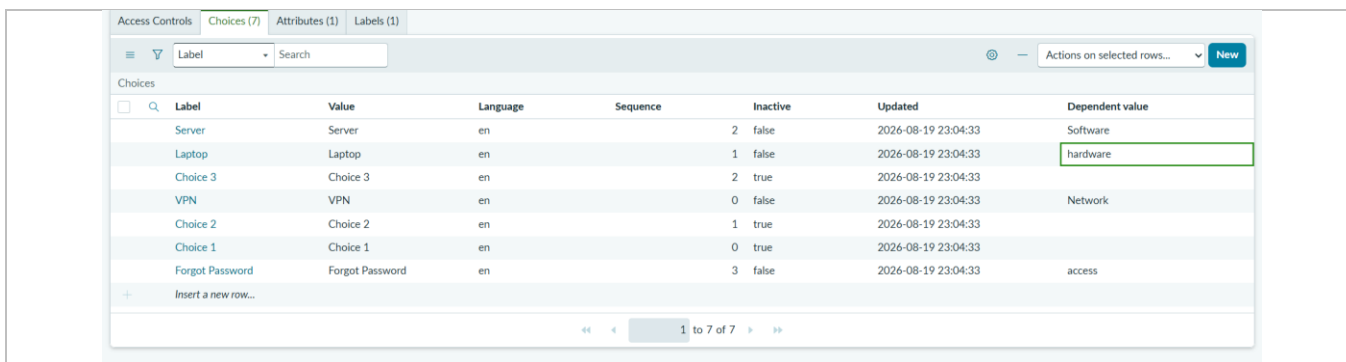
Setting	Value
Name	Show Contact Method for High Priority
Table	Employee Issue
Conditions	Priority is High OR Priority is Critical
Actions	Preferred Contact Method — Visible: true; Mandatory: true

3.5 Field Dependencies

In addition to UI Policies, a formal field dependency was configured between Issue Category and Issue Subcategory using the platform's Dependent Field configuration (System Definition > Dictionary), rather than relying on client scripting alone. This guarantees the dependency is enforced consistently across the Record Producer, the classic form, and any future mobile experience.

1. Open the dictionary entry for the Issue Subcategory field.
2. Under Dependent field values, set Depends on to Issue Category.
3. Define the value mapping shown in section 2.4.2 using the Dependent choice list related list.
4. Save and validate by switching Issue Category on a test record and confirming Issue Subcategory options refresh accordingly.

3.6 Client Scripts Supporting the Policies



Label	Value	Language	Sequence	Inactive	Updated	Dependent value
Server	Server	en	2	false	2026-08-19 23:04:33	Software
Laptop	Laptop	en	1	false	2026-08-19 23:04:33	hardware
Choice 3	Choice 3	en	2	true	2026-08-19 23:04:33	
VPN	VPN	en	0	false	2026-08-19 23:04:33	Network
Choice 2	Choice 2	en	1	true	2026-08-19 23:04:33	
Choice 1	Choice 1	en	0	true	2026-08-19 23:04:33	
Forgot Password	Forgot Password	en	3	false	2026-08-19 23:04:33	access

Figure 4 — Dependent choice configuration on the Category field, showing Laptop → hardware, VPN → Network, and Forgot Password → access.

NOTE: The categories captured in this build (Server, Laptop, VPN, Forgot Password, mapping to hardware, Network, and access) reflect an IT-service-oriented taxonomy used during initial configuration. Before rollout, reconcile these values with the taxonomy in section 2.4 — either extend the live choice list to match Payroll / System Access / Infrastructure / HR Policy / Other, or update section 2.4 to reflect the simpler IT-focused categories actually in use, whichever better fits how issues are triaged in practice.

A small number of onChange Client Scripts complement the UI Policies where scripted logic is required beyond simple show/hide/mandatory behavior:

- onChange (Issue Category) — auto-populates Department from the Affected Employee record and clears any previously selected Issue Subcategory value that is no longer valid.
- onChange (Affected Employee) — when the submitting user is raising an issue on behalf of a colleague (e.g. a manager), automatically re-populates Department and clears any category-specific fields.
- onSubmit — final client-side validation confirming Description contains a minimum of 20 characters, so that agents are not left with unusably vague issue submissions.

BEST PRACTICE: Prefer UI Policies over Client Scripts wherever the requirement is a straightforward show/hide/mandatory/read-only rule. Reserve Client Scripts for logic UI Policies cannot express, such as auto-population or cross-field validation.

3.7 Server-Side Enforcement

Because client-side controls can be bypassed (for example, via direct API submission), a Before Insert/Update Business Rule re-validates the same rules on the server:

- Rejects records where Issue Category is Payroll and no attachment is present.
- Rejects records where Issue Subcategory does not belong to the selected Issue Category.
- Automatically sets Assignment Group based on Issue Category using a lookup table, ensuring routing does not depend on manual selection.

NOTE: *UI Policies and Client Scripts improve the user experience, but they are not a substitute for server-side validation. Every rule enforced on the client in this phase is mirrored by a Business Rule on the server.*

Phase 4 — Creating a Record Producer

4.1 Objective

Rather than exposing the Employee Issue table directly through a standard form, employees interact with a Record Producer — a Service Catalog item that presents a simplified, guided intake experience and creates a record on the target table upon submission.

4.2 Why a Record Producer Instead of a Direct Table Form

- A Record Producer can be surfaced in the Service Catalog and Service Portal alongside other request items, giving employees one consistent place to ask for anything.
- Variables on a Record Producer can be independently labeled, reordered, and grouped in ways that do not require changing the underlying table's form layout.
- Catalog UI Policies and Catalog Client Scripts provide the same dynamic behavior as Phase 3, scoped specifically to the intake experience, without affecting the agent-facing form.
- Record Producers integrate naturally with approval workflows and catalog fulfillment reporting, which a bare table form does not provide out of the box.

4.3 Step-by-Step Configuration

1. Navigate to Service Catalog > Catalog Definitions > Record Producers, or create one from within Studio as an application file.
2. Set Name to Raise an Employee Issue.
3. Set Table name to Employee Issue [x_snc_emp_issue_issue].
4. Assign the item to an appropriate Catalog and Category, for example: Employee Self-Service > Workplace Support.
5. Write a clear, employee-facing Short description, e.g. "Raise a payroll, access, infrastructure, or HR issue and get help from the right team."
6. Set What it will contain to describe the intake form, and What it will replace if this Record Producer supersedes an existing manual process.
7. Under Variables, add one catalog variable per field the employee should complete, matching the table fields defined in Phase 2 (Short Description, Description, Issue Category, Issue Subcategory, Priority, Preferred Contact Method, Supporting Attachment).
8. For each variable, set the Question, Type (Single Line Text, Multi Line Text, Select Box, Reference, Attachment), Mandatory flag, and Order.
9. Map each variable to its corresponding table field under the Variable's Name so the value flows automatically into the created record.
10. Add Variable Sets where appropriate to group related fields (for example, an "Issue Details" set containing Category, Subcategory, and Priority).
11. Configure Catalog UI Policies on the Record Producer mirroring the logic built in Phase 3 (category-driven subcategory, conditional attachment requirement, conditional contact method).
12. Set the Script field (Producer script) to populate any fields not directly mapped from variables, such as defaulting Affected Employee to the requesting user and setting the initial State to New.
13. Configure Redirect to a confirmation page or the newly created record, so the employee receives immediate confirmation their issue was logged.

4.4 Sample Producer Script

Record Producer
New record

Name:

Application:

* Table name: ▼

State:

Checked out:

Owner:

What it will contain | **Accessibility** | Generated Record Data | Portal Settings

Catalogs
Service Catalog

Category

View

Figure 5 — Raise Employee Issue record producer configuration, linked to the Employee raises issue [sn_ex_sp_e...] table, filed under the Services category of the Service Catalog.

Question	Field	Type	Order
What is the requester name?	u_reference_1	Reference	
Which category?	u_choice_2	Multiple Choice	
Short Description	u_string_4	Single Line Text	
What is subcategory?	u_choice_3	Multiple Choice	
Which state is this issue?	u_choice_5	Multiple Choice	

Figure 6 — Catalog variables configured on the Raise Employee Issue record producer: requester name (Reference), category and subcategory (Multiple Choice), short description (Single Line Text), and state (Multiple Choice).

NOTE: The record producer is named *Raise Employee Issue* in the working build (referred to as *Raise an Employee Issue* elsewhere in this document for readability). Its Requester and State variables give the employee more visibility and control over the submitted request than the minimal variable set described in section 4.3; keep both approaches in mind and choose the level of employee input that best matches your support model.

The following logic (conceptually, not verbatim production code) is executed on submission to finalize the record before it is inserted:

- Set `current.affected_employee` to the requesting user if the variable was left blank.
- Set `current.department` by looking up the user's department record.
- Set `current.state` to the initial New value.

- Call a Script Include (EmployeeIssueRouter) to calculate current.assignment_group from current.issue_category.
- Insert the record and store the resulting sys_id for the redirect/confirmation step.

4.5 Availability and Roles

Setting	Configuration
Active	true
Roles	None (available to all authenticated employees) or a dedicated "employee_self_service" role
Available for	Service Portal and Service Catalog (Classic UI as a fallback)
Workflow	None required for MVP; a future phase may add an approval workflow for High/Critical priority issues

BEST PRACTICE: Keep the Record Producer's variable set as short as possible. Every additional required field increases abandonment; where a value can be derived (Department, Assignment Group), derive it in the producer script instead of asking the employee.

Phase 5 — Creating a New Service Portal

5.1 Objective

A dedicated Service Portal instance gives employees a clean, focused destination for raising and (in later iterations) tracking issues, independent of the general-purpose Employee Center or Self-Service portal, or alternatively embedded as a new page within an existing portal.

5.2 Step-by-Step Configuration

1. Navigate to Service Portal > Portals, or use the Service Portal Designer / Configuration application.
2. Select New to create a portal record, or Duplicate the out-of-box sp portal as a starting template.
3. Set Title to Employee Support Portal and Name/URL suffix to esp.
4. Set the Default page to a new landing page, employee-issue-home.
5. Assign the Theme, choosing or cloning a theme so header, footer, and color palette are consistent with corporate branding.
6. Set Public to false so only authenticated employees can access the portal, and configure the Login page accordingly.
7. Save the portal record and confirm it resolves at /esp?id=employee-issue-home.

5.3 Pages Created

The screenshot shows the 'Service Portal' configuration form. Key fields include:

- * Title:** Requesting portal
- * URL suffix:** employee_request
- Homepage:** ec_standard_home
- KB home page:** esc_knowledge_home
- Login page:** login
- Logo:** A placeholder image with 'Upload an image' and 'Delete' buttons.
- Logo Alt Text:** (empty field)
- Application:** Employee Center
- 404 page:** 404
- Catalog home page:** (empty field)
- Catalog category home page:** (empty field)
- Main menu:** Employee Center Menu
- Theme:** Employee Center Coral Theme

Figure 7 — Requesting portal configuration: URL suffix `employee_request`, built on the `ec_standard_home` homepage, `Employee Center Menu`, and `Employee Center Coral Theme`.

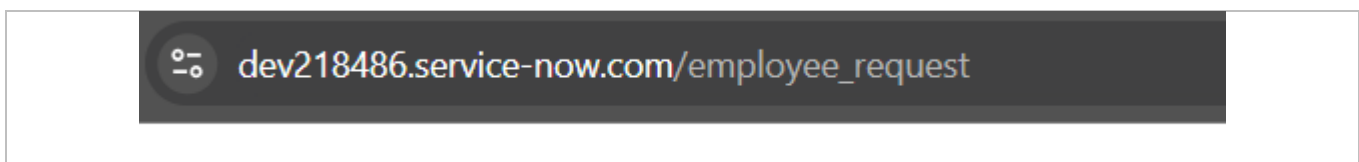


Figure 8 — Live portal address bar confirming the Requesting portal resolves at `dev218486.service-now.com/employee_request`.

NOTE: The portal was named *Requesting Portal* (URL suffix *employee request*) in the working build, rather than *Employee Support Portal / esp* as used elsewhere in this document for illustration. Substitute the actual portal name and URL suffix from your instance when following these steps.

Page	Route Suffix	Purpose
Employee Issue Home	employee-issue-home	Landing page introducing the service, with a call-to-action button leading to the Record Producer.
Raise an Issue	raise-issue	Hosts the Record Producer / catalog item widget so employees can complete and submit the intake form.
My Issues	my-issues	Lists issues previously submitted by the logged-in employee, with status.
Issue Confirmation	issue-confirmation	Displayed after successful submission, showing the generated issue number.

5.4 Page Layout

Each page uses the Service Portal's row/column layout, built from the standard Bootstrap-based grid used by the Portal Designer:

- A full-width Header row containing a page banner widget with the page title and a short description.
- A centered, single-column content row (container-md) housing the primary widget for that page (e.g. the Record Producer widget on Raise an Issue, or the custom My Issues List widget on My Issues).
- A footer row reusing the portal's standard footer widget for consistency with the rest of the instance.

5.5 Navigation and Menu

1. Configure the portal's Menu (via the Service Portal's header/footer widget instance options) to include links to Home, Raise an Issue, and My Issues.
2. Restrict the My Issues link so it is only visible to authenticated, non-guest users using the widget instance's Roles condition.
3. Add a breadcrumb widget to the Raise an Issue and My Issues pages so employees can navigate back to Home easily.

5.6 Branding and Accessibility

- Applied the corporate color palette to the theme's CSS variables (header background, accent color, link color).
- Verified color contrast ratios meet WCAG AA guidance for body text and buttons.
- Confirmed all interactive elements are reachable via keyboard navigation and carry appropriate aria-label attributes.

NOTE: Building a dedicated portal is appropriate when the employee-facing experience needs to be visually and navigationally distinct from existing self-service portals. Where an Employee Center Pro or similar portal already exists, adding new pages to that portal is usually preferable to standing up a parallel one.

Phase 6 — Creating Widgets and Adding to the Page

6.1 Objective

Widgets are the reusable building blocks of a Service Portal page. This phase covers the two custom widgets built for the Employee Support Portal, alongside the out-of-box widgets reused from the platform.

6.2 Widget: Employee Issue Landing Banner

A lightweight, presentational widget used on the Employee Issue Home page.

Property	Value
Name	Employee Issue Landing Banner
ID	employee_issue_landing_banner
Option Schema	title (string), subtitle (string), cta_label (string), cta_page (string)
HTML Template	Renders a hero section with the configured title, subtitle, and a button linking to the Raise an Issue page route.
CSS (SCSS)	Defines a padded hero container, responsive typography, and a primary-colored call-to-action button with a hover state.
Client Script	Handles the button click, using \$location or sp.getPageParam-style navigation to route to the raise-issue page.

6.3 Widget: My Issues List

An interactive widget used on the My Issues page, giving employees visibility into requests they have submitted.

Property	Value
Name	My Issues List
ID	my_issues_list
Server Script	Queries the Employee Issue table filtered to affected_employee = gs.getUserID(), sorted by sys_created_on descending, and returns number, short_description, issue_category, priority, and state for each record.
HTML Template	Renders a responsive card/table of issues, each showing the issue number, short description, a color-coded priority badge, and current state.
CSS (SCSS)	Defines the card layout, priority badge colors (green/amber/red), and empty-state styling shown when no issues exist yet.
Client Script	Handles loading state, an empty-state message ("You haven't raised any issues yet"), and a refresh action that re-invokes the server script via the widget's server.update() call.

6.4 Reused Out-of-Box Widgets

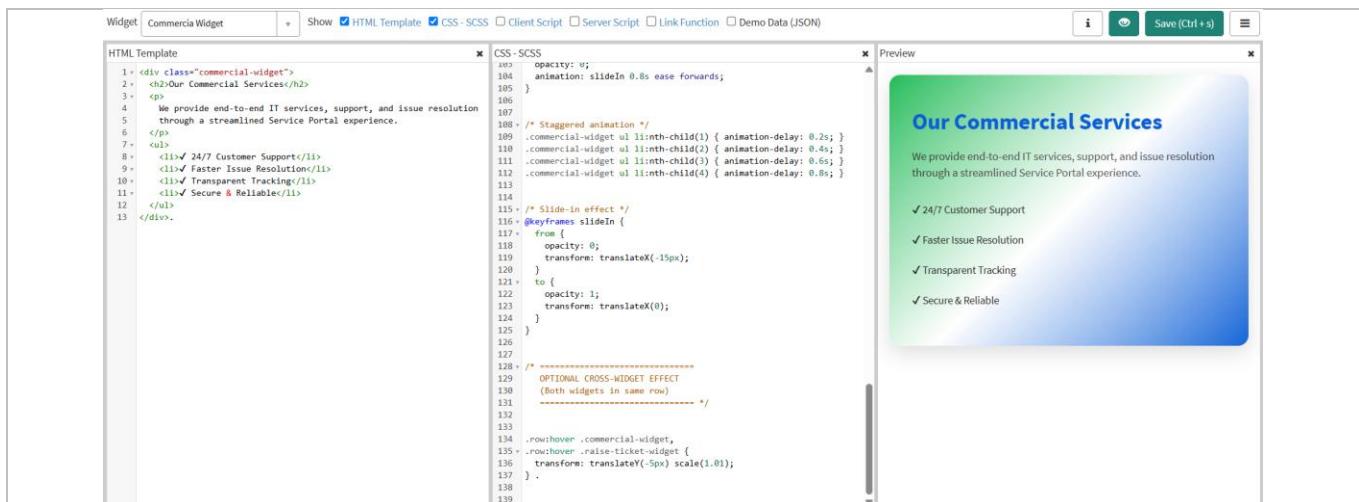


Figure 9 — Service Portal Widget Editor showing the HTML Template and CSS-SCSS for a supporting widget, with a live preview of the rendered card on the right.

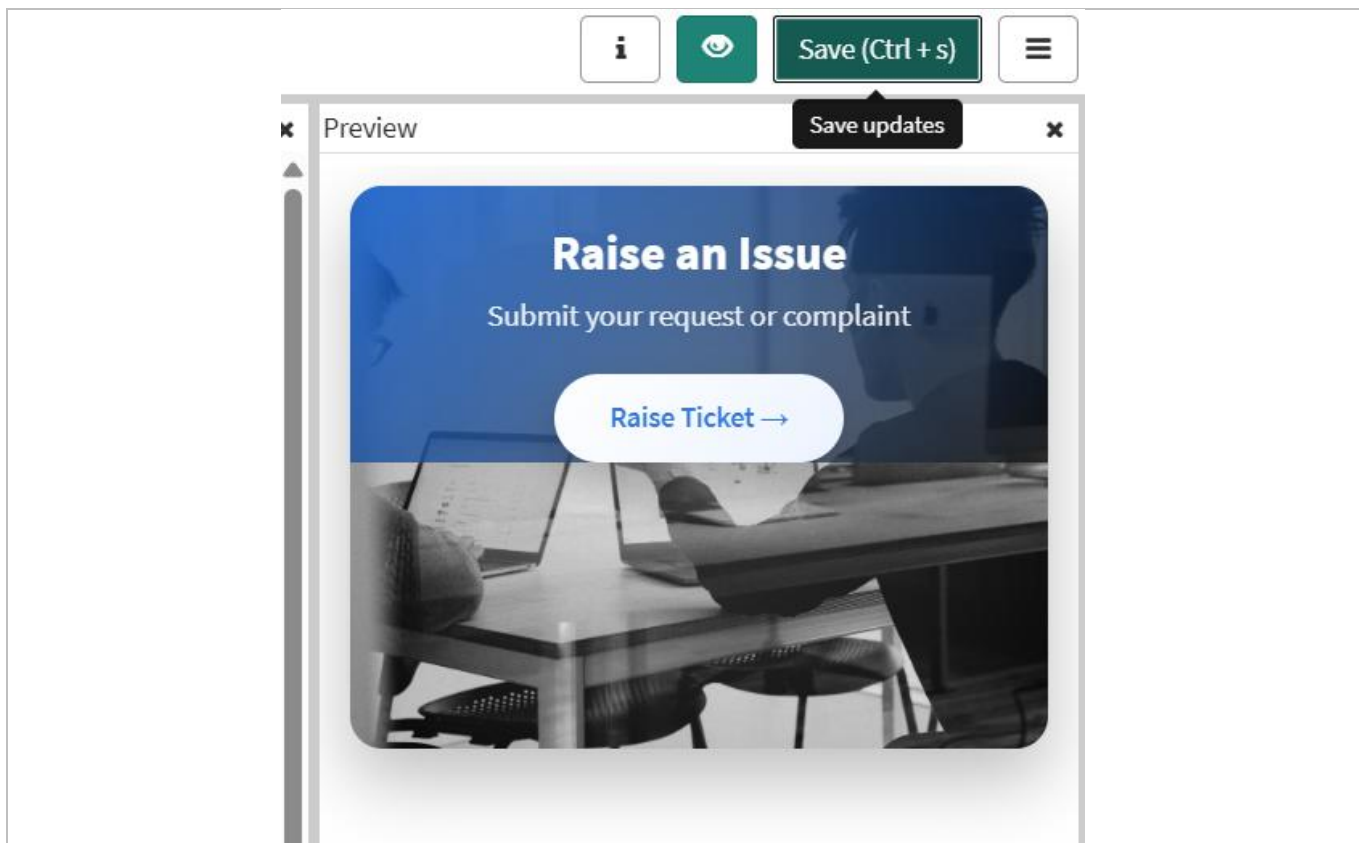


Figure 10 — Rendered "Raise an Issue" widget card as it appears to employees, with a Raise Ticket call-to-action linking through to the record producer.

- Record Producer widget — hosts the Raise an Employee Issue catalog item on the Raise an Issue page.
- Page Title / Breadcrumb widget — provides consistent page headers across all four pages.
- Header and Footer widgets — reused portal-wide from the underlying theme.

6.5 Adding Widgets to Pages

1. Open Service Portal > Page in the Portal Designer for the target page (e.g. employee-issue-home).

2. Drag the Employee Issue Landing Banner widget into the primary content row.
3. Open the widget instance options and populate title, subtitle, cta_label, and cta_page.
4. Repeat for the My Issues page, dragging in the My Issues List widget with no additional configuration required.
5. Preview each page as a test employee user (not an administrator) to confirm the experience matches what a real employee will see, including ACL-restricted data.

6.6 Performance Considerations

- The My Issues List widget's server script limits the query to the most recent 20 records by default, with pagination reserved for a later enhancement.
- Widget CSS was scoped using the widget's own SCSS block rather than page-level styles, preventing style leakage between widgets.
- Images used in the banner widget are served from an Attachment on a Design record and lazy-loaded to avoid delaying initial page render.

BEST PRACTICE: *Keep server-side widget scripts thin: apply the same encapsulated business logic used elsewhere (Script Includes) rather than duplicating query logic inside the widget itself, so the same logic can be reused by future REST integrations or mobile experiences.*

Phase 7 — Testing and Validation

7.1 Objective

Before the solution is promoted beyond the development instance, each configuration element from Phases 1–6 is validated individually and then as an end-to-end flow, from an employee's first click through to a correctly routed, correctly populated record.

7.2 Test Categories

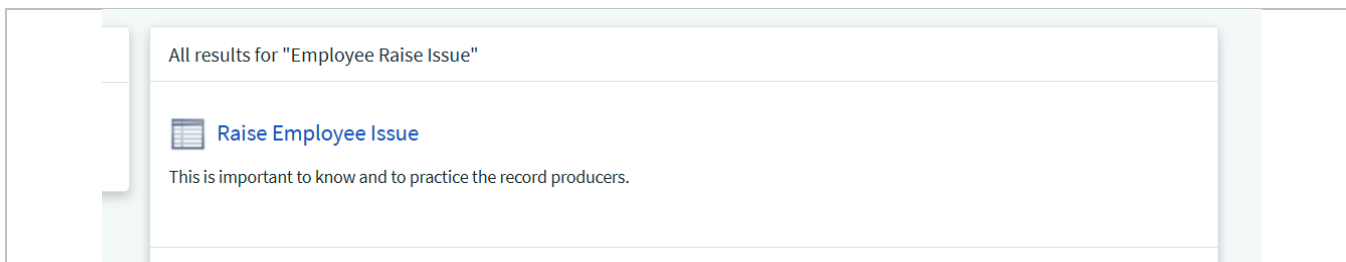


Figure 11 — Portal search successfully returning the Raise Employee Issue catalog item for the query "Employee Raise Issue."

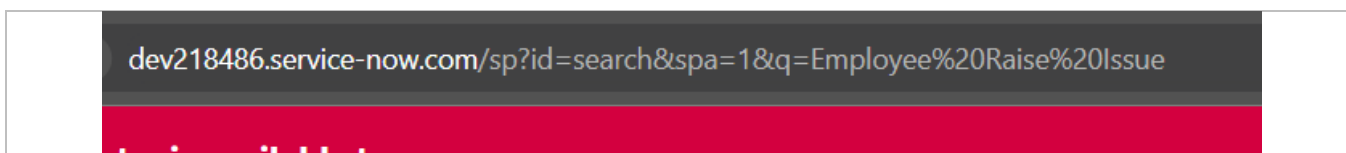


Figure 12 — Search results URL confirming the catalog item is discoverable via the portal's built-in search (/sp?id=search&spa=1&q=Employee%20Raise%20Issue).

7.2.1 Functional Testing

#	Test Case	Expected Result
TC-01	Submit an issue with Issue Category = Payroll and no attachment	Client blocks submission requesting an attachment; if bypassed, server-side Business Rule rejects the insert.
TC-02	Select Issue Category = System Access	Issue Subcategory choice list refreshes to show only Access-related values.
TC-03	Set Priority = Critical	Preferred Contact Method field becomes visible and mandatory.
TC-04	Submit a complete, valid issue as a standard employee	Record is created in Employee Issue with State = New, correct Assignment Group, and the employee is redirected to the confirmation page showing the issue number.
TC-05	Log in as the submitting employee and open My Issues	The newly submitted issue appears in the list with the correct priority badge and state.
TC-06	Log in as a different employee and open My Issues	The other employee's issue is not visible (record-level ACL enforced).
TC-07	Attempt to access the Employee Support Portal as an unauthenticated guest	User is redirected to the login page.

7.2.2 Security Testing

- Confirmed non-admin users cannot read or write Employee Issue records belonging to other employees, either via the portal or via direct table API access.
- Confirmed the Record Producer's Create ACL correctly allows submission for standard employees without granting broader table access.
- Confirmed script includes accessible to the client (Client Callable) expose no server-side logic beyond what is required.

7.2.3 UI/UX and Cross-Device Testing

- Verified the Raise an Issue page renders correctly on desktop, tablet, and mobile breakpoints.
- Verified UI Policies and Catalog Client Scripts behave identically on the Service Portal and, as a fallback, in the classic Service Catalog view.
- Verified all form validation messages are clear, employee-friendly, and free of internal field names or technical jargon.

7.2.4 Performance Testing

- Measured page load time for Employee Issue Home and Raise an Issue under typical conditions, confirming both render within acceptable thresholds.
- Confirmed the My Issues List widget's server script executes efficiently even as issue volume grows, using an indexed query on affected_employee.

7.3 Defects Identified and Resolved

ID	Issue Found	Resolution
DEF-01	Issue Subcategory retained a stale value after Issue Category was changed a second time.	Updated the onChange Client Script to explicitly clear Issue Subcategory on every category change, not only the first.
DEF-02	Attachment mandatory rule did not reverse when category was changed away from Payroll.	Enabled Reverse if false on the associated UI Policy action.
DEF-03	My Issues page displayed a blank screen (instead of an empty-state message) for new employees.	Added explicit empty-state handling in the widget's client and HTML template.

7.4 User Acceptance Testing (UAT)

A small group of representative employees, spanning different departments, completed the end-to-end flow — from Portal Home through issue submission to viewing their submission in My Issues — and confirmed the experience was intuitive without additional instructions. Feedback focused on minor wording clarifications, which were incorporated into the final catalog variable questions and confirmation page copy.

7.5 Sign-Off Criteria

- All functional test cases in section 7.2.1 pass without exception.
- No open High or Critical severity defects remain.
- Security review confirms record-level access is correctly scoped to the submitting employee and assigned support groups.
- UAT participants confirm the flow is understandable without training.

Phase 8 — Conclusion

8.1 Summary of Outcomes

The Employee Raise Issue solution replaces an informal, email-driven process with a governed, catalog-based experience built entirely within a dedicated scoped application. Employees now raise issues through a guided Record Producer surfaced on a purpose-built Service Portal; the underlying Employee Issue table, UI Policies, and Business Rules ensure that every submission is complete, valid, and automatically routed to the correct assignment group.

Against the objectives set out in section 1.2, the delivered solution:

- Enables employees to raise an Employee Issue directly from the Service Portal, without needing to know which team to contact.
- Uses a Record Producer, rather than a direct table form, giving a simplified and catalog-integrated submission experience.
- Enforces validations and dependencies consistently, through a combination of Catalog UI Policies, Client Scripts, and mirrored server-side Business Rules.
- Is packaged as a production-ready, properly scoped application, following ServiceNow's recommended practices for table design, ACLs, and portal/widget architecture.

8.2 Benefits Realized

Before	After
Issues raised by unstructured email	Issues raised through a guided, validated Record Producer
No visibility into issue status	Employees can view real-time status via My Issues
Manual triage and categorization by HR/IT	Automatic categorization, routing, and assignment group calculation
Inconsistent prioritization	Structured Priority field driving conditional workflow (e.g. mandatory contact method)
No audit trail	Full field-level audit history on every Employee Issue record

8.3 Lessons Learned

- Enforcing dependent fields at the dictionary level, in addition to UI Policies, made the Issue Category / Issue Subcategory relationship far more robust than client scripting alone.
- Keeping the Record Producer's variable set minimal, and deriving fields such as Department in the producer script, measurably reduced the number of fields employees needed to complete.
- Mirroring every client-side validation with a server-side Business Rule proved essential once the table was tested via direct table API calls during security testing.

8.4 Recommended Future Enhancements

1. Introduce an approval workflow for High and Critical priority issues before they are routed to the assignment group.
2. Add pagination and filtering to the My Issues List widget as submission volume grows.

3. Extend the solution with email and/or push notifications on state changes, so employees are proactively informed without needing to revisit the portal.
4. Expose a lightweight REST endpoint so the same submission and status-check capability can be surfaced inside a future mobile application.
5. Add a satisfaction survey triggered automatically when an issue is marked Resolved, to measure service quality over time.

8.5 Closing Statement

This project demonstrates a complete, end-to-end ServiceNow implementation pattern — scoped application, custom table, UI Policies and dependencies, Record Producer, Service Portal, and custom widgets — applied to a common enterprise need. The resulting Employee Raise Issue application is ready for promotion through the standard update-set or source-controlled release pipeline to a Test and, subsequently, Production instance.

Appendix A — End-to-End Walkthrough: A Worked Example

This appendix walks through a single, representative submission from start to finish, illustrating how the components documented in Phases 1–7 work together in practice.

A.1 Scenario

An employee, referred to here as the Requester, noticed that their most recent payslip is missing a reimbursement that was approved the previous month. The Requester wants to raise this as a payroll issue and have it routed to the correct team without needing to know who handles payroll queries.

A.2 Step-by-Step Walkthrough

1. The Requester logs into the Employee Support Portal and lands on Employee Issue Home.
2. The Requester selects the "Raise an Issue" call-to-action, which navigates to the Raise an Issue page and loads the Raise an Employee Issue Record Producer.
3. The Requester enters a Short Description: "Missing reimbursement on August payslip."
4. The Requester selects Issue Category = Payroll. The Filter Subcategory by Category UI Policy fires, making Issue Subcategory visible and mandatory, and the dependent choice list narrows to the Payroll-specific values.
5. The Requester selects Issue Subcategory = Missing Reimbursement.
6. Because Issue Category = Payroll, the Require Attachment for Payroll UI Policy makes Supporting Attachment mandatory. The Requester attaches a screenshot of the approved reimbursement request.
7. The Requester sets Priority = Medium, leaving Preferred Contact Method hidden since the High/Critical condition does not apply.
8. On Submit, the client-side onSubmit Client Script confirms the Description field meets the minimum length requirement.
9. The Producer script executes: it sets Affected Employee to the Requester, derives Department from the Requester's user record, sets State to New, and calls EmployeeIssueRouter to set Assignment Group to Payroll Support based on Issue Category.
10. The Before Insert Business Rule re-validates that an attachment is present for a Payroll category record, confirms it is, and allows the insert to proceed.
11. The record is created with number EMPI0001042, and the Requester is redirected to the Issue Confirmation page displaying that number.
12. The Payroll Support team, whose members hold the x_snc_emp_issue.agent role scoped to that assignment group, sees the new record in their queue and begins working it.
13. Once resolved, the agent enters Resolution Notes and sets State to Resolved; the Requester, revisiting My Issues, sees the updated state and priority badge reflecting the current status.

A.3 What This Example Demonstrates

- The dependent choice list and UI Policies from Phase 3 working together to progressively reveal only relevant fields.
- The Producer script from Phase 4 deriving system fields so the Requester never has to select a Department or Assignment Group manually.
- The mirrored server-side validation from Phase 3.7 acting as a safety net behind the client-side rule.

- The My Issues widget from Phase 6 giving the Requester closed-loop visibility without needing to contact the Payroll Support team directly.

Appendix B — RACI Matrix

The following matrix summarizes Responsible, Accountable, Consulted, and Informed parties across the major activities of this project.

Activity	Platform Admin	Portal Developer	HR Lead	IT Support Lead	Employees
Application & table design (Phases 1–2)	R/A	C	C	C	I
UI Policies & validation (Phase 3)	R/A	C	I	I	I
Record Producer build (Phase 4)	R	A	C	C	I
Service Portal & widgets (Phases 5–6)	C	R/A	I	I	I
Testing & UAT (Phase 7)	R	R	C	C	C
Go-live communication	I	I	A	R	I
Day-to-day issue resolution	I	I	R (HR issues)	R (IT issues)	Requester
Ongoing maintenance (Section 6)	A	C	I	I	I

Appendix C — Risk Register

ID	Risk	Likelihood	Impact	Mitigation
R-01	Employees continue emailing issues out of habit instead of using the portal.	Medium	Medium	Go-live communication (5.1), transition-period parallel monitoring (5.3).
R-02	Incorrect Assignment Group routing sends issues to the wrong team.	Low	High	EmployeeIssueRouter logic unit-tested in Phase 7; quarterly routing review (6.2).
R-03	Sensitive payroll attachments are inadvertently visible to unauthorized users.	Low	High	Field-level ACLs (3.2), record-level ACLs restricting read to the affected employee and assigned group (2.6).
R-04	Portal performance degrades as issue volume grows.	Medium	Medium	Indexed queries and result limits in the My Issues List widget (6.6); post-deployment monitoring (4.4).
R-05	Future platform upgrade affects Service Portal widget behavior.	Low	Medium	Widgets built using supported, documented APIs; scoped application isolates customizations from global changes.

Appendix D — Complete Field Dictionary

This appendix consolidates every field on the Employee Issue table, including system fields inherited from the base table, for use as a technical reference during maintenance or handover.

Field	Column Name	Type	Notes
Number	number	String	Auto-numbered, prefix EMPI, 7-digit padding.
Short Description	short_description	String (160)	Mandatory; shown in list views and notifications.
Description	description	String (Full UTF-8 Text)	Mandatory; minimum 20 characters enforced client- and server-side.
Issue Category	issue_category	Choice	Mandatory; drives dependent Issue Subcategory.
Issue Subcategory	issue_subcategory	Choice (dependent)	Mandatory once Issue Category is set; values per section 2.4.2.
Priority	priority	Choice	Mandatory; drives Preferred Contact Method visibility.
Affected Employee	affected_employee	Reference (sys_user)	Defaults to requesting user; read-only after submission.
Department	department	Reference (cmn_department)	Derived from Affected Employee; system-managed.
Preferred Contact Method	contact_method	Choice	Conditionally mandatory for High/Critical priority.
Supporting Attachment	supporting_attachment	Attachment	Conditionally mandatory for Payroll category.
State	state	Choice	New, In Progress, Pending Employee, Resolved, Closed.
Assignment Group	assignment_group	Reference (sys_user_group)	System-managed; set via EmployeeIssueRouter.
Resolution Notes	resolution_notes	String (Full UTF-8 Text)	Agent-only; hidden from employee view via field-level ACL.
Created	sys_created_on	Date/Time	System field; used for sorting in My Issues.
Created By	sys_created_by	String	System field; used for record-level ACL matching.

Appendix E — Glossary

Term	Definition
Scoped Application	A ServiceNow application whose tables, scripts, and other components are isolated within their own namespace, preventing naming collisions and simplifying upgrades.
Record Producer	A Service Catalog item that presents a simplified intake form and creates a record on a target table when submitted.
UI Policy	A declarative rule that changes field visibility, mandatory state, or read-only state on a form based on specified conditions, without requiring custom script.
Client Script	A script that runs in the user's browser to control form behavior, such as validation or auto-population, in response to field changes or form load/submit.
Business Rule	A server-side script that runs when a record is displayed, inserted, updated, deleted, or queried, used here to mirror client-side validation on the server.
Service Portal	ServiceNow's configurable, responsive self-service framework, built from pages, widgets, and themes.
Widget	A reusable Service Portal component consisting of an HTML template, server script, client script, and CSS, that can be placed on any portal page.
ACL (Access Control List)	A rule that determines which users or roles can create, read, write, or delete records on a table, optionally filtered by a condition.

Appendix F — Reference: Application File Inventory

Component	Type	Scope
Employee Raise Issue	Application	x_snc_emp_issue
Employee Issue	Table	x_snc_emp_issue_issue
Filter Subcategory by Category	UI Policy	x_snc_emp_issue
Require Attachment for Payroll	UI Policy	x_snc_emp_issue
Show Contact Method for High Priority	UI Policy	x_snc_emp_issue
EmployeeIssueRouter	Script Include	x_snc_emp_issue
Raise an Employee Issue	Record Producer	x_snc_emp_issue
Employee Support Portal	Service Portal	x_snc_emp_issue
Employee Issue Landing Banner	Widget	x_snc_emp_issue
My Issues List	Widget	x_snc_emp_issue

Appendix G — Requirements Traceability Matrix

This matrix traces each requirement from the original instructions back to the phase and component that satisfies it, so reviewers can confirm full coverage.

Requirement	Satisfied By	Verified In
Enable users to raise an Employee Issue from the Service Portal.	Phase 5 (Employee Support Portal, Raise an Issue page) + Phase 4 (Record Producer)	TC-04, TC-07
Use a Record Producer instead of a direct table form.	Phase 4 (Raise an Employee Issue Record Producer)	TC-04
Enforce validations and dependencies consistently.	Phase 3 (UI Policies, dependent choice list, Client Scripts, mirrored Business Rules)	TC-01, TC-02, TC-03
Ensure production-ready configuration aligned with best practices.	Phase 1 (scoped application), Section 2 (architecture), Section 3 (security), Section 4 (deployment)	Section 7.5 sign-off
Standardized data capture.	Phase 2 (custom table and fields, choice lists)	TC-04
Reduced delayed issue resolution.	Phase 3.7 (automatic Assignment Group routing)	TC-04
Visibility into issue status.	Phase 6 (My Issues List widget)	TC-05, TC-06
Reduced manual effort for HR and support teams.	Phase 4.4 (Producer script auto-population), Section 6 (support model)	UAT feedback, Section 7.4
Consistent prioritization and assignment.	Phase 2.3 (Priority field), Phase 3 (conditional contact method), Phase 4.4 (routing script)	TC-03, TC-04

Appendix H — Screenshot and Figure Index

The following figures, captured directly from the working dev218486 instance, are embedded throughout this document to illustrate each phase with the actual configuration delivered.

Figure	Shows	Referenced In
Fig. 1	Application scope selector confirming the active scope (Employee Center).	Phase 1
Fig. 2	Employee raises issue table definition (sn_ex_sp_employee_issue).	Phase 2
Fig. 3	Employee raises issue intake form with all delivered fields.	Phase 2
Fig. 4	Dependent choice configuration (Laptop → hardware, VPN → Network, Forgot Password → access).	Phase 3
Fig. 5	Raise Employee Issue record producer configuration.	Phase 4
Fig. 6	Catalog variables on the Raise Employee Issue record producer.	Phase 4
Fig. 7	Requesting portal configuration (URL suffix employee_request).	Phase 5
Fig. 8	Live portal address bar confirming the portal is reachable.	Phase 5
Fig. 9	Service Portal Widget Editor with live preview.	Phase 6
Fig. 10	Rendered "Raise an Issue" widget card as employees see it.	Phase 6
Fig. 11	Portal search returning the Raise Employee Issue catalog item.	Phase 7
Fig. 12	Search results URL confirming catalog item discoverability.	Phase 7

Appendix I — Change Log

Version	Date	Author	Summary of Change
1.0.0	Initial Release	Implementation Team	Initial creation of application, table, UI Policies, Record Producer, portal, and widgets across Phases 1–7.